

Documentación Técnica De Clientrack: Aplicación Móvil Para La Gestión De Clientes Y Casos

Yiseth Karina Gomez Hernandez

Universidad EAN

Desarrollo Web

Diego Fernando Chinchilla Beltrán

3 de noviembre de 2026

ClienTrack: aplicación móvil para la gestión de clientes y casos

Este documento describe el desarrollo de ClienTrack, una aplicación para Android que funciona como un sistema básico de gestión de relaciones con clientes. La aplicación se construyó como proyecto final de la asignatura Desarrollo Web y reúne los temas vistos durante el curso: diseño de interfaces, validación de datos, manejo de eventos y almacenamiento de información en una base de datos.

La idea surgió de un problema común en los negocios pequeños. Muchos emprendedores y vendedores independientes atienden a sus clientes por WhatsApp, llamadas o correo, y llevan los pendientes de memoria o en notas sueltas. Con el tiempo se olvidan solicitudes, se pasan fechas de entrega y no queda claro qué cliente está esperando una respuesta. Herramientas como Salesforce o HubSpot resuelven este problema, pero su costo y su complejidad no se justifican para un negocio que apenas empieza. ClienTrack toma la parte más útil de esas herramientas, que es el seguimiento de casos por cliente, y la lleva a una aplicación sencilla que funciona en el celular sin conexión a internet.

En las siguientes secciones se presentan los objetivos y el alcance del proyecto, los requisitos funcionales y técnicos, la arquitectura de la aplicación, el diseño de la base de datos, la lógica de las funciones principales, la interfaz de usuario y las pruebas realizadas. Al final se explica cómo instalar y ejecutar el proyecto.

Objetivos

Objetivo general

Desarrollar una aplicación móvil para Android que permita a un usuario registrar a sus clientes y hacer seguimiento a los casos de cada uno, con prioridad, estado y fecha límite, guardando la información en una base de datos local SQLite.

Objetivos específicos

1. Implementar el registro y el inicio de sesión de usuarios con validación de datos y almacenamiento seguro de contraseñas.
2. Diseñar una base de datos relacional con tablas para usuarios, clientes y casos.
3. Permitir crear, consultar, editar y eliminar clientes y casos.
4. Facilitar la consulta de la información por medio de búsqueda, filtros por estado y un resumen de los casos pendientes y vencidos.
5. Construir una interfaz clara que oriente al usuario desde la primera pantalla.

Alcance

La versión entregada incluye una pantalla de bienvenida con instrucciones de uso, el registro y el inicio de sesión con sesión persistente, la gestión completa de clientes y de casos, la búsqueda en tiempo real, el filtro por estado, un resumen con contadores y la detección automática de casos vencidos. Toda la información se guarda en el mismo dispositivo.

Quedan por fuera de esta versión la sincronización en la nube, el acceso desde varios dispositivos, las notificaciones de fechas límite, la recuperación de contraseña por correo y el trabajo de varios usuarios sobre los mismos clientes. Estas funciones se plantean como mejoras en las conclusiones.

Requisitos funcionales

La Tabla 1 resume las funciones que cumple la aplicación. Cada requisito corresponde a uno de los flujos que se muestran en el video de demostración: registro, inicio de sesión, lista, creación, detalle, edición, eliminación y navegación entre pantallas.

Tabla 1

Requisitos funcionales de ClienTrack

Código	Requisito	Descripción
RF-01	Bienvenida	Mostrar una pantalla que explique qué hace la aplicación y cómo se usa.
RF-02	Registro	Crear una cuenta con nombre, correo y contraseña de al menos seis caracteres, con confirmación.
RF-03	Inicio de sesión	Validar el correo y la contraseña contra la base de datos y mantener la sesión abierta.
RF-04	Gestión de clientes	Agregar, consultar, buscar, editar y eliminar clientes.
RF-05	Crear caso	Registrar un caso asociado a un cliente con asunto, descripción, prioridad, estado y fecha límite.
RF-06	Lista de casos	Mostrar los casos ordenados por prioridad y fecha, con búsqueda y filtro por estado.
RF-07	Detalle del caso	Ver toda la información del caso y los datos de contacto del cliente.
RF-08	Editar y cerrar	Modificar un caso existente y marcarlo como cerrado.
RF-09	Eliminar	Eliminar un caso o un cliente, con confirmación previa.
RF-10	Resumen	Contar los casos abiertos, en proceso, cerrados y vencidos.
RF-11	Cerrar sesión	Salir de la cuenta y volver al inicio de sesión.

Requisitos técnicos

El proyecto se desarrolló en Android Studio con el lenguaje Java. Para la interfaz se usaron archivos XML y la librería Material Components de Google, y para los datos se utilizó SQLite, que viene incluida en Android y no necesita un servidor. En la Tabla 2 se detallan las herramientas y sus versiones. Se eligieron versiones estables y no versiones beta, para que el proyecto se pueda compilar sin problemas en otro equipo.

Tabla 2

Herramientas y versiones utilizadas

Herramienta	Versión	Uso en el proyecto
Android Studio	Koala 2024.1.1 o superior	Entorno de desarrollo
Java	17	Lenguaje de la lógica de la aplicación
JDK de Gradle	21	Ejecución de Gradle
Gradle	8.7	Compilación del proyecto (se descarga con el wrapper)
Android Gradle Plugin	8.5.2	Integración de Gradle con Android
SDK de Android	compileSdk y targetSdk 34; minSdk 26	Funciona desde Android 8.0
AppCompat	1.6.1	Compatibilidad entre versiones de Android
Material Components	1.11.0	Botones, campos de texto, chips, calendario y diálogos
RecyclerView	1.3.2	Listas de casos y de clientes
ConstraintLayout	2.1.4	Organización de vistas en pantalla
SQLite	Incluida en Android	Base de datos local
JUnit / Espresso	4.13.2 / 3.5.1	Pruebas unitarias y pruebas de interfaz

Nota. Gradle 8.7 solo es compatible con Java hasta la versión 21. Si Android Studio pide seleccionar la JVM de Gradle al abrir el proyecto, se debe elegir la versión 21.

Arquitectura de la aplicación

El código se organizó por capas, de manera que cada parte tenga una sola responsabilidad. Las pantallas se encargan de recibir las acciones del usuario y mostrar los resultados, mientras que todas las consultas a la base de datos están en una sola clase. Esta separación sigue la recomendación de la guía de arquitectura de Android de no mezclar la interfaz con el acceso a los datos. La Tabla 3 describe los paquetes del proyecto.

Tabla 3

Paquetes del proyecto

Paquete	Clases	Responsabilidad
ui	WelcomeActivity, RegisterActivity, LoginActivity, MainActivity, ClientesActivity, CasoFormActivity, CasoDetalleActivity	Pantallas: reciben eventos, validan y muestran resultados
adapter	CasoAdapter, ClienteAdapter	Convertir las listas de datos en filas del RecyclerView
model	Usuario, Cliente, Caso	Representar los datos de cada tabla
data	DatabaseHelper	Crear las tablas y ejecutar todas las consultas SQL
util	SessionManager, Validador, Formatos, UiUtils	Sesión, validaciones, cifrado, fechas y colores

Nota. Todas las clases están dentro del paquete co.edu.ean.clientrack.

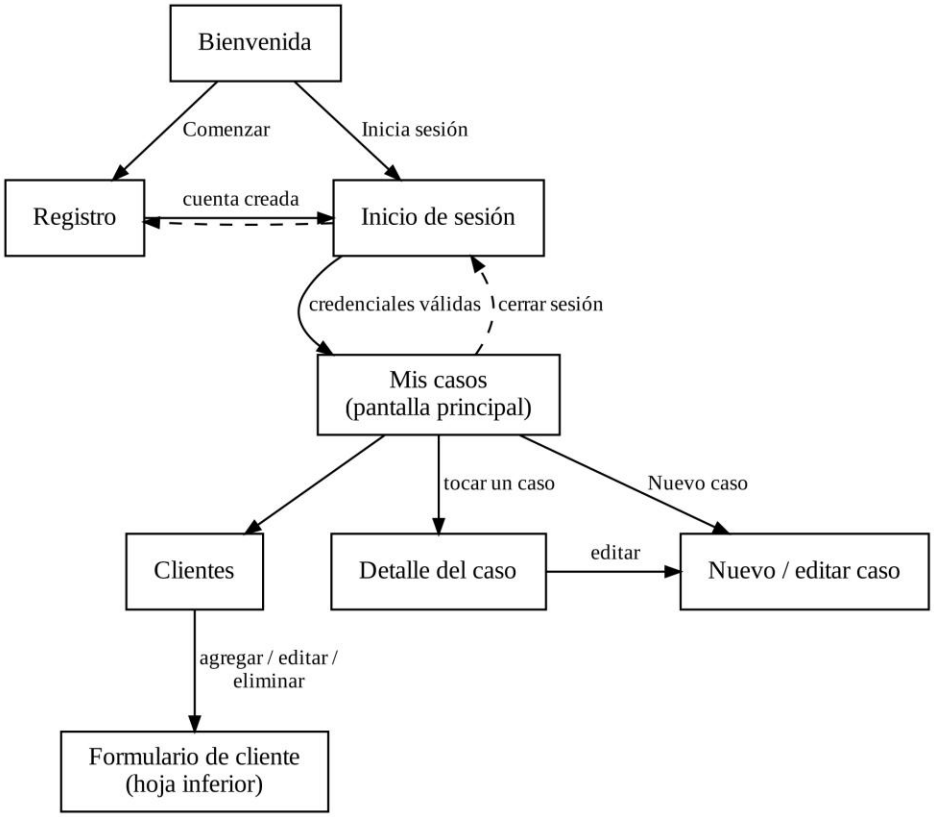
El diseño de cada pantalla está en la carpeta res/layout, y los colores, estilos y textos en res/values. Así, los colores de la aplicación se cambian en un solo archivo, colors.xml, de forma parecida a como se usan las variables en CSS. Las pantallas nunca escriben SQL directamente: siempre llaman a un método de DatabaseHelper, por ejemplo, buscarCasos o insertarCliente.

Flujo de navegación

La Figura 1 muestra cómo se pasa de una pantalla a otra. La aplicación abre en la bienvenida y desde ahí el usuario puede crear una cuenta o iniciar sesión. Una vez adentro, la pantalla principal es el punto central desde el que se llega a los clientes, al formulario de casos y al detalle de cada caso.

Figura 1

Flujo de navegación entre pantallas



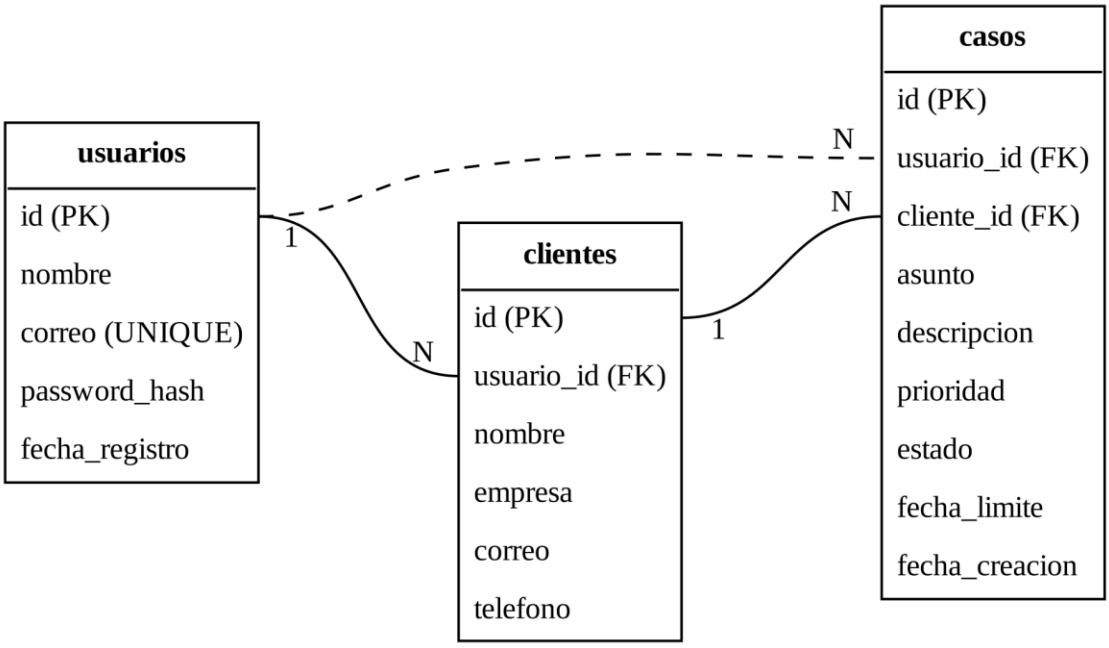
Nota. Las líneas punteadas indican regresos: cerrar sesión y volver al inicio de sesión desde el registro.

Base de datos

La aplicación usa una base de datos SQLite llamada clientrack.db, que se crea en el dispositivo la primera vez que se abre la aplicación. La clase DatabaseHelper extiende SQLiteOpenHelper, que es la forma que recomienda Android para crear y abrir este tipo de bases de datos (Android Developers, s.f.-b). La base de datos tiene tres tablas relacionadas, como se observa en la Figura 2.

Figura 2

Modelo entidad-relación de la base de datos



Nota. PK = llave primaria; FK = llave foránea. La línea punteada representa la relación directa entre usuarios y casos.

Relaciones entre tablas

Un usuario puede tener muchos clientes, y cada cliente puede tener muchos casos. La tabla casos también guarda el usuario_id, aunque este dato se podría obtener a través del cliente. Se tomó esta decisión para que todas las consultas filtren directamente por el usuario que inició sesión, y así ninguna persona pueda ver la información de otra.

Las llaves foráneas tienen la regla ON DELETE CASCADE, de modo que al eliminar un cliente también se eliminan sus casos. SQLite no activa las llaves foráneas por defecto (SQLite, s.f.), por lo que se activan en el método onConfigure con setForeignKeyConstraintsEnabled(true). Antes de borrar un cliente, la aplicación le indica al usuario cuántos casos se van a eliminar con él.

Descripción de las tablas

Las Tablas 4, 5 y 6 describen las columnas de cada tabla, su tipo de dato y sus restricciones.

Tabla 4

Estructura de la tabla usuarios

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador del usuario
nombre	TEXT	NOT NULL	Nombre completo
correo	TEXT	NOT NULL, UNIQUE	Correo de acceso; no se puede repetir
password_hash	TEXT	NOT NULL	Contraseña cifrada con SHA-256
fecha_registro	TEXT	NOT NULL	Fecha de creación de la cuenta

Tabla 5

Estructura de la tabla clientes

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador del cliente
usuario_id	INTEGER	NOT NULL, FK a usuarios(id), ON DELETE CASCADE	Usuario dueño del cliente
nombre	TEXT	NOT NULL	Nombre del cliente
empresa	TEXT	Ninguna	Empresa (opcional)
correo	TEXT	Ninguna	Correo de contacto (opcional)
telefono	TEXT	Ninguna	Teléfono de contacto (opcional)

Tabla 6

Estructura de la tabla casos

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador del caso
usuario_id	INTEGER	NOT NULL, FK a usuarios(id), ON DELETE CASCADE	Usuario dueño del caso
cliente_id	INTEGER	NOT NULL, FK a clientes(id), ON DELETE CASCADE	Cliente al que pertenece
asunto	TEXT	NOT NULL	Título del caso
descripcion	TEXT	Ninguna	Detalle del caso (opcional)
prioridad	TEXT	NOT NULL, CHECK (Alta, Media, Baja)	Prioridad del caso
estado	TEXT	NOT NULL, CHECK (Abierto, En proceso, Cerrado)	Estado del caso
fecha_limite	TEXT	NOT NULL	Fecha en que se debe resolver
fecha_creacion	TEXT	NOT NULL	Fecha en que se registró

Nota. Las fechas se guardan en formato año-mes-día (por ejemplo, 2026-11-03).

Decisiones de diseño

Las restricciones CHECK hacen que la base de datos solo acepte prioridades y estados válidos. La aplicación ya limita estas opciones con botones de selección, así que se trata de una doble validación: si por un error del código llegara un valor distinto, la base de datos lo rechaza.

Las fechas se guardan como texto en formato año-mes-día porque, en ese formato, el orden alfabético coincide con el orden cronológico. Esto permite ordenar los casos por fecha y saber si un caso está vencido comparando directamente el texto de la fecha límite con la fecha de hoy. Además, se crearon índices sobre clientes.usuario_id, casos.usuario_id y casos.cliente_id, que son las columnas que más se usan en las consultas.

Lógica principal

Registro e inicio de sesión

Cuando el usuario se registra, la aplicación valida los campos y revisa con el método existeCorreo que el correo no esté registrado. Luego convierte la contraseña en un resumen SHA-256 con la clase MessageDigest de Java y guarda el usuario con registrarUsuario. De esta forma, la contraseña nunca queda guardada en texto plano. Al terminar, la aplicación vuelve al inicio de sesión con el correo ya escrito.

En el inicio de sesión se calcula el resumen SHA-256 de la contraseña escrita y se busca un usuario que tenga ese correo y ese resumen. Si existe, su identificador y su nombre se guardan en SharedPreferences mediante la clase SessionManager, de manera que la sesión se mantiene aunque se cierre la aplicación. Todas las consultas posteriores usan ese identificador.

Manejo de eventos

Todos los formularios siguen el mismo ciclo que se trabajó en clase con JavaScript, adaptado a las clases de Android:

1. **Acción:** el usuario toca un botón, por ejemplo Guardar.
2. **Escucha:** el método setOnClickListener detecta el evento, como lo hace addEventListener en JavaScript.
3. **Lógica:** se validan los datos. Si hay un error, el mensaje aparece debajo del campo correspondiente con TextInputLayout.setError.
4. **Mutación:** los datos se guardan en SQLite y la pantalla se actualiza con un mensaje de confirmación y la lista recargada, sin reiniciar la aplicación.

Búsqueda, filtros y orden

La lista de casos se obtiene con el método buscarCasos, que une las tablas casos y clientes con un JOIN para mostrar el nombre del cliente en cada fila. La búsqueda usa LIKE sobre el asunto, el nombre del cliente y la empresa, y se ejecuta con cada letra que el usuario escribe gracias a un TextWatcher. Los caracteres especiales % y _ se escapan para que se busquen como texto normal y no como comodines.

Los filtros por estado agregan una condición a la consulta, y los resultados se ordenan primero por prioridad (Alta, Media y Baja) y después por la fecha límite más cercana. La lista se vuelve a cargar cada vez que la pantalla principal aparece de nuevo (método onResume), así siempre refleja los cambios hechos en otras pantallas.

Resumen y casos vencidos

El método obtenerResumen calcula en una sola consulta cuántos casos hay abiertos, en proceso, cerrados y vencidos, usando SUM(CASE WHEN ...). Un caso se considera vencido cuando no está cerrado y su fecha límite es anterior a la fecha actual. Estos casos se marcan en rojo en la lista y en el detalle.

Gestión de clientes

La lista de clientes se obtiene con un LEFT JOIN y la función COUNT, para mostrar cuántos casos tiene cada cliente. Se usa LEFT JOIN y no JOIN para que también aparezcan los clientes que todavía no tienen casos. El formulario para crear, editar o eliminar un cliente se abre en una hoja inferior, y la lista tiene su propia búsqueda por nombre o empresa que ignora mayúsculas y tildes. Como todo caso debe pertenecer a un cliente, si el usuario intenta crear un caso sin tener clientes, la aplicación se lo indica y lo lleva a la pantalla de clientes.

Validaciones

La Tabla 7 resume las reglas de validación de cada formulario. Los mensajes de error aparecen debajo del campo que tiene el problema y el formulario no se guarda hasta que todos los datos sean correctos.

Tabla 7

Reglas de validación por pantalla

Pantalla	Campo	Regla
Registro	Nombre	Obligatorio; solo letras y espacios; mínimo 3 caracteres
Registro	Correo	Obligatorio; formato válido; no puede estar registrado
Registro	Contraseña	Mínimo 6 caracteres
Registro	Confirmación	Debe ser igual a la contraseña
Inicio de sesión	Correo y contraseña	Obligatorios; deben coincidir con un usuario registrado
Cliente	Nombre	Obligatorio; solo letras y espacios; mínimo 3 caracteres
Cliente	Correo	Opcional; si se escribe, debe tener formato válido
Cliente	Teléfono	Opcional; solo números, entre 7 y 10 dígitos
Caso	Cliente	Obligatorio; debe existir al menos un cliente
Caso	Asunto	Obligatorio; mínimo 3 caracteres
Caso	Fecha límite	Obligatoria; en un caso nuevo no puede ser anterior a hoy
General	Eliminar y cerrar sesión	Se pide confirmación antes de realizar la acción

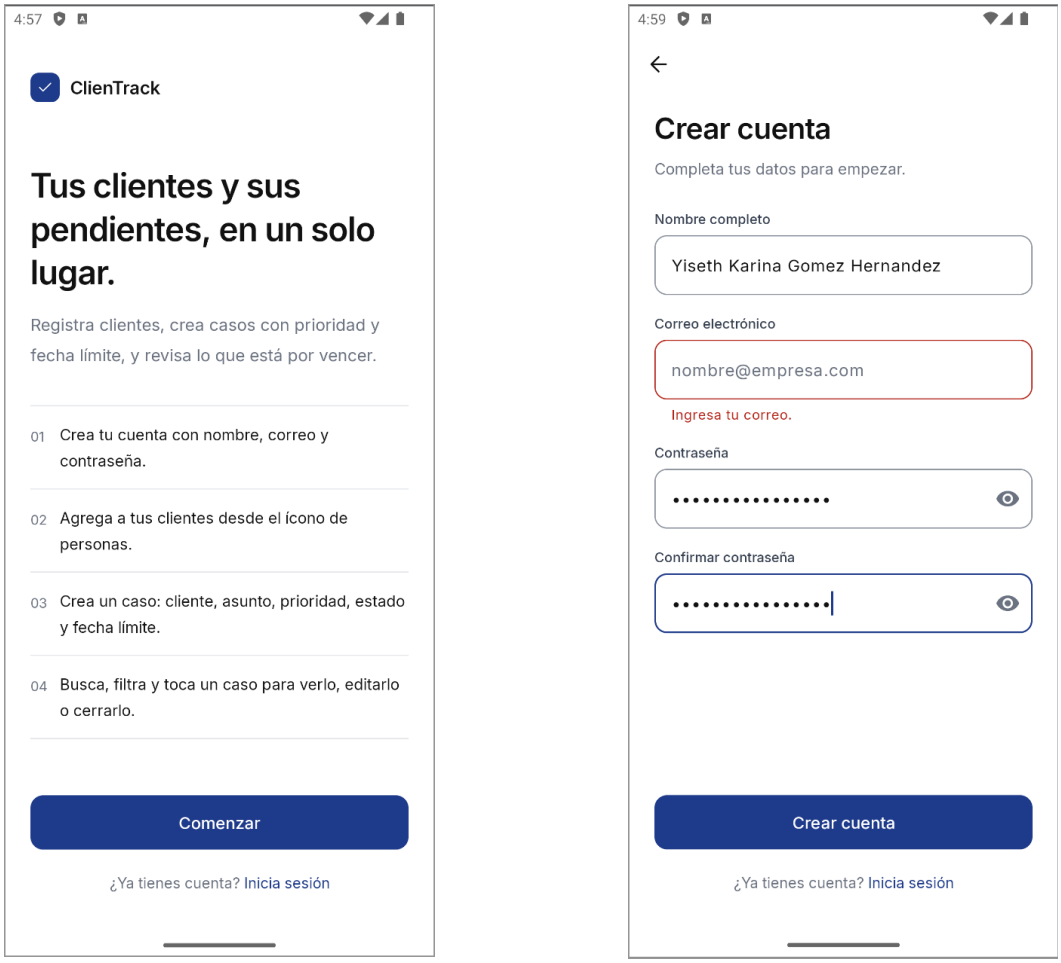
Interfaz de usuario

La interfaz sigue un estilo sencillo: fondo blanco, texto casi negro, grises neutros y un solo color de acento azul. La jerarquía de la información se marca con el tamaño y el peso del texto y con el espacio entre elementos, en lugar de usar muchos colores. Se utilizó el tema Material 3 de Google (Google, s.f.) y la tipografía Inter (Andersson, s.f.), que se incluye en el proyecto con su licencia.

El estado de cada caso se muestra con una etiqueta de color: azul para abierto, violeta para en proceso, gris para cerrado y rojo para vencido. Todos los elementos que se pueden tocar miden al menos 48 dp, que es el tamaño mínimo recomendado para usarlos cómodamente con el dedo. Las Figuras 3 a 7 muestran las principales pantallas de la aplicación.

Figura 3

Pantallas de bienvenida y registro

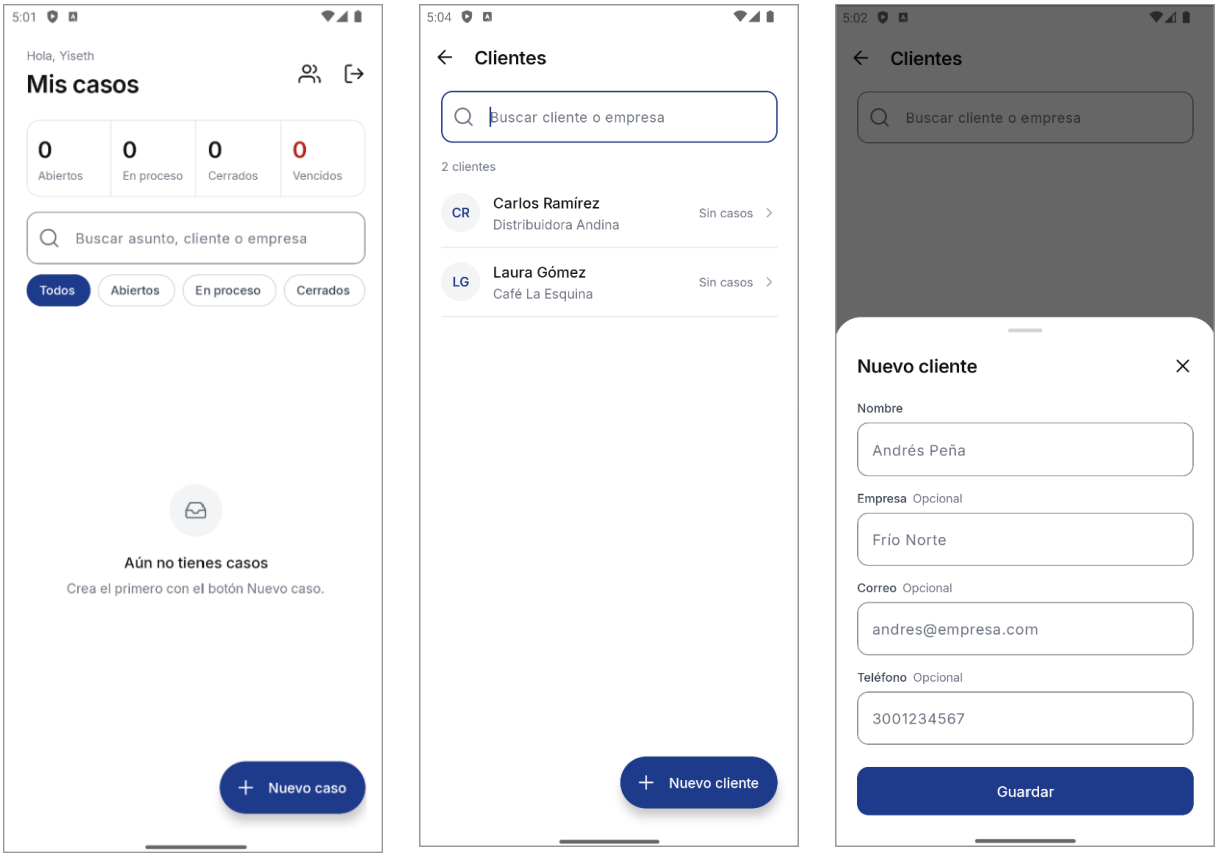


a) Bienvenida b) Registro

Nota. En la pantalla de registro se observa el mensaje de error que aparece cuando el correo está vacío.

Figura 4

Pantalla principal y gestión de clientes

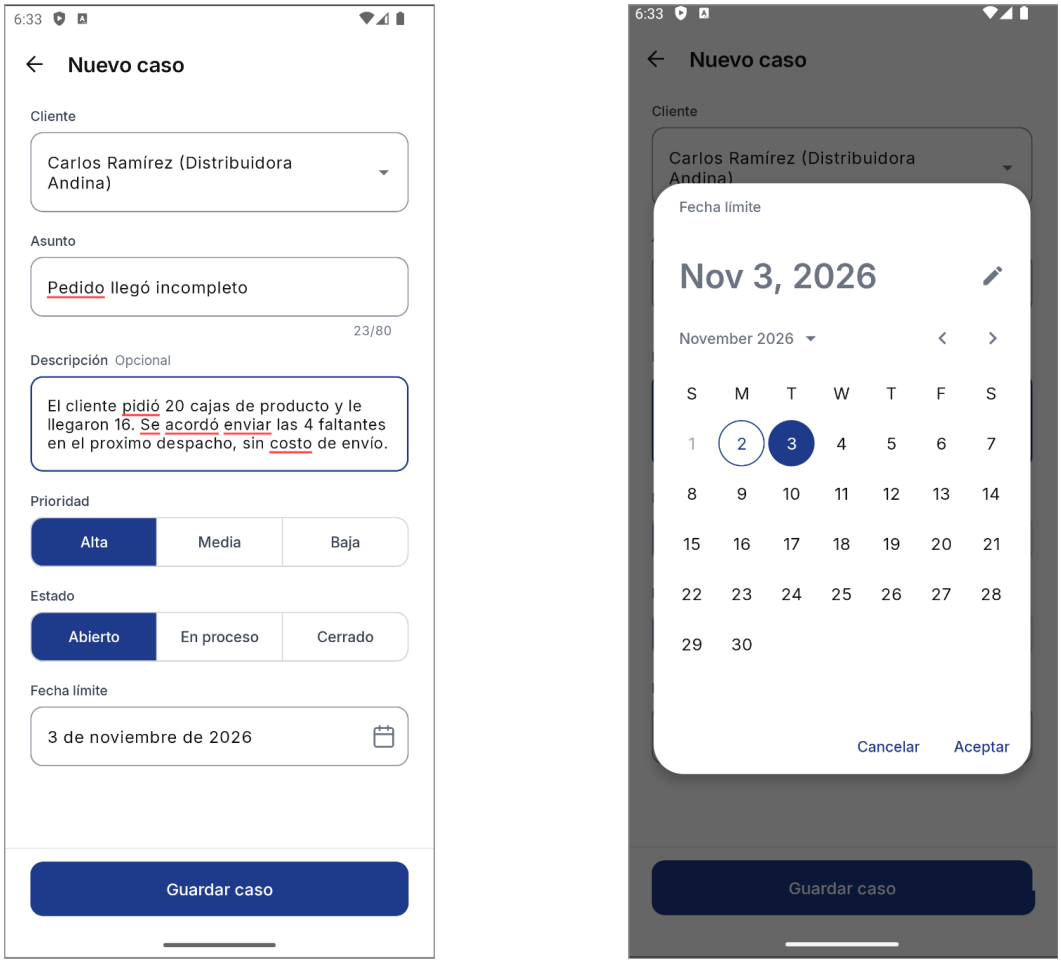


a) Mis casos b) Clientes c) Nuevo cliente

Nota. La pantalla principal se muestra sin casos registrados. El formulario de clientes se abre como una hoja inferior.

Figura 5

Formulario para crear un caso



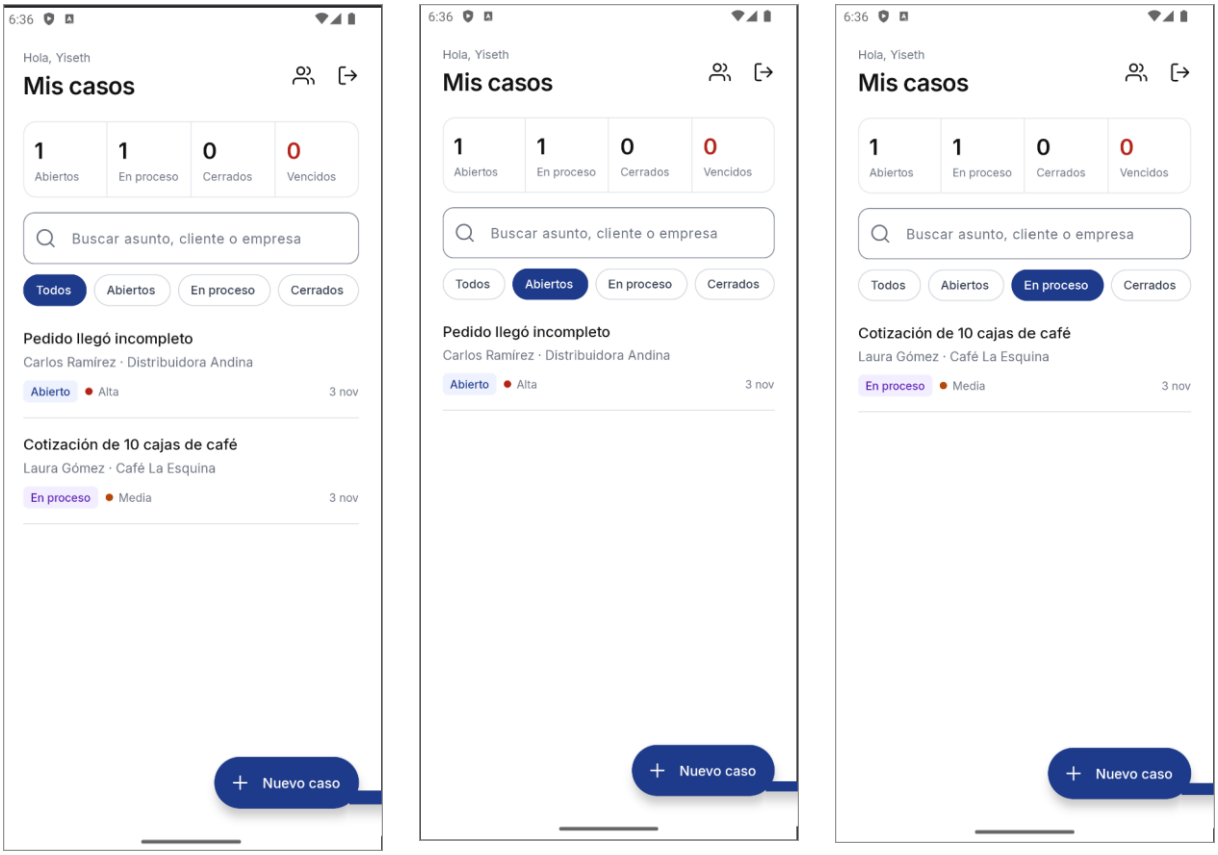
a) Nuevo caso

b) Selección de la fecha límite

Nota. La prioridad y el estado se eligen con botones de selección. La fecha límite se escoge en un calendario que, para casos nuevos, no permite fechas anteriores a hoy.

Figura 6

Lista de casos y filtros por estado

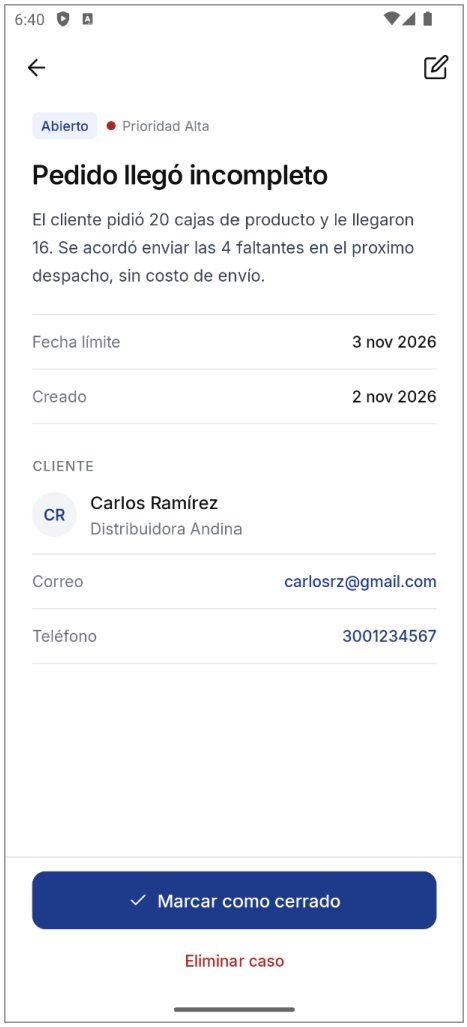


a) Todos b) Abiertos c) En proceso

Nota. Los contadores de la parte superior se actualizan al crear o modificar un caso. Cada fila muestra el estado, la prioridad y la fecha límite.

Figura 7

Detalle de un caso



Nota. El ícono de lápiz de la parte superior abre el formulario de edición. El correo y el teléfono del cliente se pueden tocar para escribirle o llamarlo, y la opción Eliminar caso pide confirmación antes de borrar.

Pruebas

Además de probar manualmente cada flujo en el emulador (Medium Phone, API 34), se escribieron pruebas automáticas para las partes más importantes. La Tabla 8 resume qué se probó.

Tabla 8

Pruebas realizadas

Tipo	Ubicación	Qué se probó
Unitarias (JUnit)	app/src/test	Validaciones de correo, teléfono, nombre, asunto y contraseña; resumen SHA-256; formatos de fecha; búsqueda sin tildes; regla de caso vencido (19 pruebas)
Interfaz (Espresso)	app/src/androidTest	Bienvenida; errores del inicio de sesión; contraseñas distintas en el registro; registro e inicio de sesión; aviso de cliente faltante; crear, cerrar y eliminar un caso (7 pruebas)
Manuales	Emulador	Todos los flujos del video: registro, inicio de sesión, lista, crear, detalle, editar, eliminar, búsqueda, filtros y cierre de sesión

Instalación y ejecución

Para ejecutar el proyecto en otro equipo se deben seguir estos pasos:

1. Descargar o clonar el repositorio y abrir la carpeta ClienTrack en Android Studio con File, Open.
2. Esperar a que termine la sincronización de Gradle. La primera vez descarga las dependencias, por lo que necesita internet.
3. Si aparece el aviso para seleccionar la JVM de Gradle, elegir la versión 21.
4. Seleccionar un emulador o un celular con la depuración USB activada y presionar Run.
5. Crear una cuenta, agregar un cliente desde el ícono de personas y crear el primer caso con el botón Nuevo caso.

Conclusiones

El desarrollo de ClienTrack permitió aplicar en un solo proyecto los temas del curso. La separación entre la interfaz, la lógica y los datos facilitó hacer cambios en el diseño sin tocar la

base de datos, y el uso de llaves foráneas con eliminación en cascada evitó que quedaran casos sin cliente. También fue importante validar los datos en dos niveles, en los formularios y en la base de datos, y guardar las contraseñas cifradas desde el principio.

Como trabajo futuro se propone sincronizar la información en la nube, por ejemplo con Firebase, para usarla desde varios dispositivos; agregar notificaciones antes de que un caso se venza; y sumar un módulo de oportunidades de venta para que la aplicación cubra también la parte comercial de un CRM.

Referencias

Andersson, R. (s.f.). *Inter* [Tipografía]. https://rsms.me/inter/

Android Developers. (s.f.-a). *Guide to app architecture*.

https://developer.android.com/topic/architecture

Android Developers. (s.f.-b). *Save data using SQLite*.

https://developer.android.com/training/data-storage/sqlite

Google. (s.f.). *Material Design 3*. https://m3.material.io/

Oracle. (s.f.). *Class MessageDigest* [Documentación de Java SE 17].

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/security/MessageDigest.html

SQLite. (s.f.). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html